

System Design for Instagram

1. Understand the Problem and Establish Design Scope

Instagram is a popular social media platform centered around **visual content sharing**. Users can upload photos and videos, connect with others by following them, and consume content through personalized feeds. Our design aims to replicate these core functionalities while ensuring the system is robust and scalable.

Functional Requirements

- **User Management:**
 - **Registration and Authentication:** Users can sign up, log in, and securely manage their profiles.
 - **Profile Management:** Users can view and edit their profile information (e.g., bio, profile picture).
- **Content Management:**
 - **Photo/Video Upload:** Users can upload single or multiple photos/videos, add captions, apply filters, and tag other users.
 - **Content Feed:** Users see a personalized feed of posts from accounts they follow.
 - **Interactions:** Users can like posts, add comments, and share posts.
 - **Stories:** Users can create temporary photo/video stories that expire after 24 hours.
- **Social Graph:**
 - **Follow/Unfollow:** Users can follow and unfollow other users.
 - **Follower/Following List:** Users can view who follows them and who they follow.
- **Discovery:**
 - **Search:** Users can search for other users, hashtags, and locations.
 - **Explore Tab:** The system suggests popular or relevant content based on user interests and engagement.
- **Direct Messaging (DM):**
 - **One-to-One Chat:** Users can send private messages (text, photos, videos) to each other.
 - **Group Chat:** Users can create and participate in group conversations.

Non-Functional Requirements

- **High Availability:** The system must be accessible almost all the time (e.g., 99.99% uptime) as users expect continuous service.
- **Scalability:** It must handle **billions of users** and **trillions of media files**. The system should scale horizontally to accommodate growth in users and data.
- **Low Latency:** Feeds should load rapidly (within hundreds of milliseconds), and content uploads should feel instantaneous.
- **Consistency:** While eventual consistency is acceptable for follower counts, core

actions like post uploads must be **immediately consistent**. Media files must be durable and never lost.

- **Reliability:** All user data, especially photos and videos, must be reliably stored and retrieved without loss.
- **Security:** User data, including private messages and login credentials, must be **encrypted** both in transit and at rest.
- **Fault Tolerance:** The system should gracefully handle failures of individual components without impacting overall service.

2. Propose High-level Design

High-level design will leverage a **microservices architecture** to ensure scalability, maintainability, and fault isolation.

API Design

Use a **RESTful API** for client-server communication, providing clear and intuitive endpoints.

All requests will go through an API Gateway.

- POST /v1/users/register: Register a new user.
- POST /v1/users/login: Authenticate and log in a user.
- POST /v1/posts: Upload a new photo/video post.
- GET /v1/feed: Retrieve the personalized feed for the authenticated user.
- POST /v1/users/{user_id}/follow: Follow a user.
- DELETE /v1/users/{user_id}/follow: Unfollow a user.
- GET /v1/posts/{post_id}: Fetch details for a specific post.
- POST /v1/posts/{post_id}/like: Like a post.
- POST /v1/posts/{post_id}/comment: Add a comment to a post.
- GET /v1/search?q={query_string}: Search for users or hashtags.
- POST /v1/stories: Upload a new story.
- POST /v1/messages: Send a direct message.

High-level System Architecture

System will be composed of several independent services communicating via APIs and a message queue:

1. **Client Applications:** Mobile apps (iOS, Android) and web clients.
2. **API Gateway:** Acts as the single entry point, handling authentication, rate limiting, and routing requests to appropriate microservices.
3. **Microservices:**
 - **User Service:** Manages user profiles, authentication, authorization, and the social graph (followers/following).
 - **Post Service:** Handles post metadata (captions, hashtags, user tags), likes, and comments. It doesn't store media directly.
 - **Media Service:** Responsible for uploading, storing, processing (resizing, filtering), and retrieving actual photo/video files.
 - **Feed Service:** Aggregates posts from followed users and generates personalized feeds.

- **Search Service:** Indexes user profiles, hashtags, and post metadata for efficient searching.
- **Story Service:** Manages the lifecycle of temporary stories.
- **Messaging Service:** Handles real-time direct messages.
- 4. **Content Delivery Network (CDN):** Stores copies of media files globally to deliver them quickly to users based on their geographic location.
- 5. **Message Queue (e.g., Apache Kafka):** Decouples services and enables asynchronous communication. Used for events like new post uploads (triggering processing, fan-out), likes, and comments.
- 6. **Databases:** Different types of databases will be used based on service needs:
 - **Relational Database (e.g., PostgreSQL, MySQL):** For structured data with strong consistency requirements, like user profiles, post metadata, and follower relationships.
 - **NoSQL Wide-Column Store (e.g., Cassandra):** Ideal for storing the social graph (follower lists, following lists) and possibly aggregated feed data due to its high write throughput and scalability.
 - **Key-Value Store (e.g., Redis):** Primarily used for caching frequently accessed data, session management, and storing personalized feeds for quick retrieval.
 - **Object Storage (e.g., AWS S3, Google Cloud Storage):** Highly scalable and durable storage for raw photo and video files.

Algorithms

- **Feed Generation (Hybrid Approach):**
 - **Fan-out on Write (Push Model):** When a user posts, the new post ID is immediately pushed to the in-memory feed (e.g., a Redis sorted set) of all their followers. This ensures low latency for reading feeds, especially for users who follow many people. This is efficient for most users.
 - **Fan-out on Read (Pull Model):** For users with an extremely high number of followers (e.g., celebrities with millions), pushing to all feeds becomes impractical. In this case, when a follower requests their feed, the system fetches posts from these high-follower accounts, combines them with pre-pushed posts, and sorts them.
 - **Relevance Ranking:** The feed will not just be chronological. A ranking algorithm will consider factors like engagement (likes, comments), recency, user relationships, and content type to prioritize posts.
- **Search:** Uses an inverted index for fast lookup of users by username, and posts by hashtags.
- **Media Processing:** Asynchronous tasks for resizing images into multiple resolutions and applying filters.

Data Model

Here are simplified data models for key services:

User Service (Relational Database) users table:

- user_id (PK, UUID)

- username (string, unique, indexed)
- email (string, unique)
- password_hash (string)
- profile_picture_url (string)
- bio (text)
- created_at (timestamp)

follows table:

- follower_id (FK to users.user_id)
- following_id (FK to users.user_id)
- created_at (timestamp)
- (Composite PK: (follower_id, following_id))

Post Service (Relational Database) posts table:

- post_id (PK, UUID)
- user_id (FK to users.user_id)
- media_url (string - URL to CDN)
- caption (text)
- created_at (timestamp, indexed for feed sorting)
- location (string, optional)

likes table:

- like_id (PK, UUID)
- post_id (FK to posts.post_id)
- user_id (FK to users.user_id)
- created_at (timestamp)
- (Composite PK: (post_id, user_id) for uniqueness)

comments table:

- comment_id (PK, UUID)
- post_id (FK to posts.post_id)
- user_id (FK to users.user_id)
- text (text)
- created_at (timestamp)

Feed Service (Redis - In-memory data structures) user_feed:{user_id}: Redis Sorted Set

- member: post_id
- score: timestamp (for sorting)

Media Service (Object Storage)

- Stores raw media files with unique keys (e.g., {post_id}.jpg, {post_id}_small.jpg).

3. Design Deep Dive

Scaling the Database

- **Sharding (Horizontal Partitioning):** This is crucial for handling massive data volumes.
 - **User Data:** Shard the users and follows tables by user_id. When a user signs up, their user_id determines which database shard their data (profile, followers, following) resides on. This keeps related data together, minimizing

cross-shard queries.

- **Post Data:** Posts can also be sharded by `user_id` for quick retrieval of all posts by a specific user. For retrieving posts across many users (e.g., for feed generation), a combination of sharding and distributed queries (or relying on the Feed Service's pre-aggregation) is needed.
- **Read Replicas:** All relational databases will have multiple read replicas to distribute query load. Writes go to the primary database, which then replicates to the read replicas.
- **NoSQL for Social Graph:** For the follows table, especially given the many-to-many relationship and high read/write volume, a **NoSQL wide-column store like Cassandra** is highly suitable. It can be partitioned by `user_id` to quickly retrieve all followers or who a user is following.
- **Database Connection Pooling:** Efficiently manage database connections to prevent overwhelming the database with too many open connections.

Caching

Caching is vital for reducing database load and improving response times.

- **CDN (Content Delivery Network):**
 - **Purpose:** Primarily caches static content like photos and videos.
 - **Mechanism:** When a user requests a media file, the request goes to the nearest CDN edge server. If cached, it's served immediately. Otherwise, the CDN fetches it from the Media Service's Object Storage, caches it, and serves it. This drastically reduces media load times.
- **In-Memory Cache (Redis):**
 - **Session Data:** Stores user session tokens for fast authentication.
 - **User Profiles:** Caches frequently accessed user profile data.
 - **Feeds:** The Feed Service stores personalized feeds for active users in Redis sorted sets, allowing for very fast retrieval.
 - **Hot Posts/Trends:** Caches popular posts or trending hashtags for the Explore tab.
 - **Caching Strategy:** We'll use a **read-through cache** for feeds (if not in Redis, generate and cache) and a **write-through/write-back cache** for user profile updates or post interactions where consistency is critical.
- **Application-Level Caching:** Services can also implement small, localized caches for frequently accessed data within their own memory.

Filter Results by Time or Business Type

- **Filtering by Time:**
 - All posts have a `created_at` timestamp.
 - When fetching a feed or search results, the Feed Service or Search Service can easily sort the results by this timestamp to show the most recent content first. Database indexes on `created_at` are essential.
 - Redis sorted sets for feeds naturally support sorting by score (timestamp).
- **Filtering by Business Type:**

- Requires an additional attribute in the users table, e.g., `account_type` (personal, business, creator).
- When searching or populating the Explore tab, the Search/Explore Service can filter or prioritize results based on this `account_type`.
- For business-specific features (e.g., contact info, analytics), a separate `business_profiles` table linked to users would store additional metadata.

Final Architecture Diagram

Flow for Post Upload:

1. **Client** uploads media and metadata to **API Gateway**.
2. **API Gateway** forwards metadata to **Post Service** and media to **Media Service**.
3. **Media Service** stores the raw media in **Object Storage** and puts a message on the **Message Queue** for asynchronous processing (resizing, applying filters).
4. **Post Service** saves post metadata (caption, tags, `media_url` from Media Service) to its **Relational DB**.
5. **Post Service** then publishes a "new post" event to the **Message Queue**.
6. **Feed Service (Fan-out service)** consumes this event and updates the Redis feeds of all followers (for the push model).
7. **Search Service** consumes the event to index the new post's content and hashtags.

Flow for Feed Retrieval:

1. **Client** requests feed from **API Gateway**.
2. **API Gateway** authenticates and forwards to **Feed Service**.
3. **Feed Service** checks the user's feed in **Redis Cache**.
4. If found (cache hit), it returns the list of `post_ids`.
5. If not found (cache miss), **Feed Service** queries **User Service** (to get following list) and **Post Service** (to get recent posts from followed users).
6. **Feed Service** aggregates, sorts, and ranks the `post_ids`, stores them in **Redis** for future requests, and returns them to the client.
7. **Client** receives `post_ids` and then requests media URLs from the **CDN** (which serves from cache or **Object Storage**).

4. Wrap Up

The proposed Instagram system design leverages a **microservices architecture** to achieve high scalability, availability, and fault tolerance. By separating concerns into distinct services (User, Post, Media, Feed, Search, Story, Messaging), we can scale each component independently based on its specific load.

Key design decisions include:

- Using an **API Gateway** for centralized request handling.
- Employing **Object Storage** and a **CDN** for efficient and fast media delivery.
- Utilizing a **Message Queue (Kafka)** for asynchronous processing and decoupling

services.

- Adopting a **polyglot persistence** approach, combining **Relational Databases** for structured, consistent data, **NoSQL databases (Cassandra)** for scalable social graph data, and **Redis** for high-speed caching and feed management.
- Implementing a **hybrid fan-out model** for feed generation to balance read and write performance for diverse user types.
- Applying **database sharding** and **read replicas** to distribute data and query load.
- Extensive **caching layers** to minimize database hits and improve user experience.

This architecture provides a robust foundation capable of handling the immense scale and real-time demands of a global social media platform like Instagram.